Exponential-Range Mass Production of Boolean Functions

Local recovery, disjoint scheduling, and bounded congestion

Samuel Schlesinger

September 2026

Abstract

We study the cost of evaluating one Boolean function on many independent inputs. For $f : \{0,1\}^n \to \{0,1\}$, let $f^{\times t}$ evaluate f on t such inputs. Uhlig showed that $t = 2^{o(n/\log n)}$ copies preserve the sharp one-copy asymptotic. We retain the optimal order of growth throughout every fixed exponential range below the 2^n-copy scale: for every fixed $0 \le \gamma < 1$,

$$\mathrm{C}(f^{\times t}) = O_\gamma(2^n/n) \qquad \text{for all } t \le 2^{\gamma n}.$$

The bound is uniform in f and t. It controls the optimal order of growth, but does not preserve Uhlig's sharp one-copy leading constant.

The proof uses local coded recovery instead of scanning the whole truth table. An evaluation code gives each requested symbol many affine-line recovery sets. A deterministic scheduler chooses disjoint lines within request groups, so each shorter resource function is queried at most once per group; induction handles the demand across groups. With $k+1$ equal input blocks this gives the copy exponents

$$\frac{1}{2}, \frac{2}{3}, \frac{3}{4}, \dots,$$

which approach one.

This work was developed with substantive AI assistance. It began in a ChatGPT.com conversation that the author exported as a LaTeX document. Later interactive sessions used Claude Fable 5 and 5.1 through Claude Code, and ChatGPT 5.6 Sol through Codex. These systems contributed to experiments, broad literature searches, and several core parts of the proof strategy and its internal constructions. The author checked the arguments and citations and is responsible for the paper.

1 Introduction

How much circuit size can be saved when the same function must be evaluated on many unrelated inputs? This is a direct-sum question in which the outputs are independent but the circuit implementing them need not be: intermediate computations may be reorganized and shared across copies.

For a Boolean function $f : \{0,1\}^n \to \{0,1\}$ and an integer $t \ge 1$, define

$$f^{\times t}(x^{(1)}, \dots, x^{(t)}) = \big(f(x^{(1)}), \dots, f(x^{(t)})\big).$$

The naive construction uses t disjoint copies of a circuit for f and gives

$$\mathrm{C}(f^{\times t}) \le t\mathrm{C}(f).$$

The mass-production problem asks how large t can become before the cost must rise above the worst-case scale for one copy. For unrestricted circuits, the naive direct-sum bound can be far from optimal. Uhlig's theorem allows $t = 2^{o(n/\log n)}$ independent evaluations with no asymptotic increase in the leading worst-case one-copy synthesis cost. In our basis and cost convention that coefficient is 1 [4]; its numerical value can depend on the basis and convention. See Uhlig's later account [12], Wegener's Theorem 10.2.3 [13], and the modern restatement in [7]; the 1974 paper [11] is a precursor.

We prove, by a different construction, that the order-of-growth plateau reaches $2^{\gamma n}$ simultaneous evaluations for every fixed $\gamma < 1$. The price is that we control only the optimal worst-case order $2^n/n$, not the sharp one-copy leading constant.

1.1 Main result

We fix the standard basis $\{\wedge, \vee, \neg\}$, with fan-in two for the binary gates. Fan-out and output designations are free, and circuit size is the number of non-input gates. Any other fixed finite complete basis changes the result by at most a constant-factor simulation.

Theorem 1.1 (Exponential-range mass production)**.** *For every fixed $0 \le \gamma < 1$, there is a constant A_γ such that, for every integer $n \ge 1$, every Boolean function $f : \{0,1\}^n \to \{0,1\}$, and every integer $1 \le t \le 2^{\gamma n}$,*

$$\mathrm{C}(f^{\times t}) \le A_\gamma \frac{2^n}{n}.$$

The order of the quantifiers matters. Once γ is fixed, one constant A_γ works for every function and every batch size in the stated range; the bound has no factor depending on t. At the largest allowed batch size, the amortized size per requested value is therefore

$$\frac{\mathrm{C}(f^{\times t})}{t} = O_\gamma\left(\frac{2^{(1-\gamma)n}}{n}\right).$$

The theorem does not make A_γ uniform as $\gamma \to 1$.

There must nevertheless be a transition before the 2^n-copy scale. If $n \ge 2$ and f depends on all n variables, every circuit for $f^{\times t}$ has size at least $tn/2$. Hence a uniform $O(2^n/n)$ plateau cannot persist once $t = \omega(2^n/n^2)$; see Section 8. Our theorem reaches every fixed exponential rate, but leaves this near-endpoint regime open.

1.2 How the construction works

Paul's general many-copy construction provides a useful contrast [9, Lemma 2]. It treats the truth table of f as an ordered database: sort the t input addresses, sweep once through the hardwired table, and restore the answers to their original order. This global lookup strategy works for arbitrary t and gives

$$O\Big(\max\{n2^n, nt\log^2 t\}\Big).$$

For every fixed $\gamma < 1$ and $t \le 2^{\gamma n}$, the full-table term $n2^n$ eventually dominates.

We instead turn the truth table into shorter resource functions and recover a requested value from a selectable local set of them. The quantity that has to be controlled is *simultaneous demand*: how many different suffixes ask for the same resource? Disjoint recovery sets control demand within a group, and recursion controls it across groups. Both Paul's construction and ours use sorting, but for different jobs. Paul sorts input addresses before a global table scan; we sort scheduling and routing records after choosing the local recovery sets.

Here is the construction in the order in which a requested value passes through it.

1. **Pack restrictions.** Split each input as $x = (a, z)$. For every fixed suffix z, pack the values $f(a, z)$ over all prefixes a into the information symbols of a constant-rate low-degree evaluation code over $\mathbb{F}_q$.
2. **Recover locally.** A requested information symbol can be reconstructed from the $q - 1$ non-target points on any affine line through it. About $q^{\ell-1}$ directions are available, so every request has many alternative recovery sets.
3. **Schedule bounded congestion.** Partition the requests into C groups. A deterministic greedy circuit chooses pairwise disjoint recovery lines inside each group. Consequently, each code coordinate is requested on at most one suffix per group and at most C suffixes overall.
4. **Recurse on the suffix.** For a fixed code coordinate, evaluating its resource function on those at most C suffixes is itself a C-copy problem, now on only $d = |z|$ variables. The recursive hypothesis pays for this remaining simultaneous demand.

Section 3 first shows the disjoint-recovery idea in Uhlig's two-copy construction. The local gadget is stated in Proposition 4.2, and Proposition 6.1 accounts for its circuit cost. The parameter choice is then isolated in Section 7. Gate-level details for direction selection and routing are deferred to Section A.

1.3 Why the exponents approach one

For an integer $k \ge 1$, let P_k be the claim that every fixed copy exponent below $k/(k+1)$ has worst-case cost $O(2^n/n)$. To prove P_k, write

$$n = (k+1)m + O_k(1)$$

and view the input as $k+1$ blocks of length about m. The outer construction encodes one block for local recovery; the remaining k blocks support the recursive resource functions.

Write $\gamma = k/(k+1) - \eta$. In the new range not already covered by P_{k-1}, choose the number of request groups by

$$\log_2 C = (k-1)m - \frac{\eta n}{2} + O(1).$$

Since $\log_2 t \le km - \eta n + O_k(1)$, each group contains at most

$$2^{m - \eta n/2 + O_k(1)}$$

requests: strictly fewer than the one-block scale 2^m. At the same time, C has exponent strictly below $(k-1)/k$ when measured against the remaining k blocks, so P_{k-1} evaluates every resource. Fixed finite-field slack makes all bookkeeping terms exponentially smaller than 2^n.

The base case P_1 uses two equal blocks and has no recursive congestion. At every later level, one block supplies the scheduling room inside each group and the preceding statement is invoked on the other blocks.

1.4 Relation to earlier work

The closest asymptotic guarantees are summarized below. Here "one-copy leading term" means the basis-dependent Shannon–Lupanov worst-case asymptotic.

Result	Copy range	Circuit-size bound	Comparison
Uhlig [12, 13, 7]	$\log t = o(n/\log n)$	one-copy leading term	optimal order and leading constant in a smaller copy range
Paul, Lemma 2 [9]	arbitrary t	$O(\max\{n2^n, nt\log^2 t\})$	$O(n2^n)$ throughout every fixed exponential range below one
Albrecht, Theorem 1 [1]	$t \le 2^n$	$O(2^n n^2)$	the full exponential range, with polynomial overhead
This paper	$t \le 2^{\gamma n}$ for fixed $\gamma < 1$	$O_\gamma(2^n/n)$	optimal worst-case order throughout every fixed exponential range

For fixed $\gamma < 1$ and $t \le 2^{\gamma n}$, Paul's second term is $o(n2^n)$, so his displayed bound is $O(n2^n)$. The theorem proved here improves that bound by a factor of n^2 at the level of polynomial prefactors. Paul's g^k is the same k-fold product task considered here; his circuits permit arbitrary binary Boolean gates, which changes our fixed basis cost by at most a constant factor.

Albrecht studies the same simultaneous-realization problem over a complete binary basis [1, Theorem 1]. His construction handles every $t \le 2^n$ with size $O(2^n n^2)$ and depth $O(n \log n)$. Since the one-copy Shannon size is $\Theta(2^n/n)$, this is an $O(n^3)$ multiplicative overhead, as Albrecht also notes. Thus his theorem covers the endpoint copy range but does not give a constant-factor plateau at the one-copy scale.

The coding interpretation begins with Uhlig's two-copy construction, which is the length-$(2^p + 1)$ single-parity-check code viewed as a two-request disjoint recovery code. Systematic multivariate polynomial evaluation, recovery on affine lines, and the use of disjoint recovery sets appear in the batch-code literature, especially Section 3.4 of Ishai–Kushilevitz–Ostrovsky–Sahai [6]. Their geometric procedure chooses lines randomly and deletes intersections. The construction here instead gives a deterministic greedy selection circuit, partitions requests into groups, recursively absorbs the resulting inter-group congestion, and charges the scheduler and fixed-wire routing explicitly. We make no priority claim for these refinements.

Later lifted-polynomial constructions also use line-based recovery to obtain batch-code guarantees [5]. Private-information-retrieval codes of Fazeli, Vardy, and Yaakobi form a related family in which each information symbol has many disjoint recovery options [3]. The self-contained proof of Theorem 1.1 uses only the elementary product-code and affine-line facts proved below.

Independently of historical priority, the technical contribution established here is a self-contained deterministic disjoint-line scheduler, an explicit fixed-wire scatter/gather implementation, and the bounded-congestion composition that drives the equal-block induction.

Machine-checked companion. An accompanying Lean 4 development formalizes the circuit model, local recovery, constructive scheduler and routing, composition bound, and equal-block induction. It proves a rational, discrete-exponent form of Theorem 1.1; the stated real-rate version follows by choosing a slightly larger rational exponent and absorbing finitely many initial input lengths. The source is available at `https://github.com/SamuelSchlesinger/algebraic-circuits`.

AI-assisted development and provenance. The project began in an interactive conversation on ChatGPT.com, exported to the author's computer as a LaTeX document. Later work used Claude Fable 5 and 5.1 through Claude Code and ChatGPT 5.6 Sol through Codex in a series of interactive sessions. These systems assisted with mathematical experiments, broad literature searches, and parts of the proof strategy and its internal constructions; their role went beyond prose editing. The author chose which directions to pursue, checked and revised the arguments and literature claims, and takes responsibility for the paper.

2 Circuit conventions and worst-case notation

Our circuits are finite directed acyclic graphs. Source vertices are input bits or the constants zero and one. Internal vertices are labelled by $\wedge$, $\vee$, or $\neg$, with the usual fan-in. A circuit for a map $F : \{0,1\}^N \to \{0,1\}^m$ has m designated output wires, which may be input wires or gate outputs and may repeat. Fan-out, constants, and output designations carry no cost; size is the number of internal gates. We write $\mathrm{C}(F)$ for the minimum size of such a circuit. All leading-constant comparisons in this paper refer to this fixed model.

Let

$$L(n) = \max_{f:\{0,1\}^n \to \{0,1\}} \mathrm{C}(f).$$

The Shannon counting lower bound and Lupanov's synthesis upper bound imply

$$L(n) = \Theta(2^n/n)$$

for every fixed finite complete basis; see [10, 8, 13]. We use only the upper bound.

For $r \geq 1$, define

$$L_r(n) = \max_{f:\{0,1\}^n \to \{0,1\}} \mathrm{C}(f^{\times r}).$$

For every $r \geq 1$, one has $L_r(n) \geq L(n)$: fix all but one of the r input blocks and retain the corresponding output. Thus $2^n/n$ is the optimal worst-case order even when several outputs are requested.

For each f, choose a circuit for $f^{\times r}$ of size at most $L_r(n)$. If a construction supplies fewer than r live inputs, the unused inputs of this circuit may be fixed arbitrarily and their outputs ignored. Thus a bound for $L_r(n)$ also applies to every smaller number of copies.

All circuits below are nonuniform. Field representations, irreducible polynomials, sorting networks, and other finite combinatorial objects may be fixed as part of the circuit for each input length.

We use $\widetilde{O}_\ell(X)$ for

$$X \cdot \mathrm{poly}_\ell(n, \log q, \log(t+2), \log(C+2)).$$

Only polynomial factors are suppressed; in the parameter regimes used below, they never affect an exponential rate.

For a positive integer k, write $[k]_0 = \{0, \dots, k-1\}$.

We repeatedly use two standard circuit primitives. First, after representing $\mathbb{F}_{2^b}$ in a polynomial basis, field addition, multiplication, inversion of a nonzero element, and comparison of canonical b-bit representations have circuits of size $\mathrm{poly}(b)$. For example, inversion can be implemented by binary exponentiation to the power $2^b - 2$. Second, R records of width w can be sorted by a fixed sorting network using $O(R \log^2 R)$ compare–exchange units, each of size $O(w)$; see [2]. Sorting by a leading type bit, with original positions used as tie breakers when needed, also gives stable fixed-wire compaction within the same asymptotic bound.

3 The two-copy construction as disjoint recovery

Uhlig's two-copy argument already contains the combinatorial invariant used later. We will not use its numerical bound. What matters is that each requested value has more than one representation, allowing the two requests to choose representations that do not reuse a resource at the same time.

Fix a prefix length p, put $M = 2^p$, and define the restrictions

$$f_i(z) = f(i, z), \qquad i \in \{0, \dots, M-1\}.$$

Uhlig's two-copy construction introduces the following functions; see the proof of Wegener's Theorem 10.2.3 [13] and the overview in [7, Section 1.2]:

$$g_0 = f_0, \qquad g_i = f_{i-1} \oplus f_i \quad (1 \le i < M), \qquad g_M = f_{M-1}.$$

Every restriction has two representations:

$$f_i = \bigoplus_{j=0}^{i} g_j = \bigoplus_{j=i+1}^{M} g_j.$$

Given requests $i \le j$, recover f_i from the prefix set $\{0, \dots, i\}$ and f_j from the suffix set $\{j+1, \dots, M\}$. The sets are disjoint, so each resource function g_j is evaluated on at most one suffix.

For two requests arriving in the opposite order, a comparator first orders their prefixes, conditional swaps route the two suffixes to the prefix and suffix recovery branches, and a final swap restores the output order. This costs only $\mathrm{poly}(n)$ gates.

The map $(f_0, \dots, f_{M-1}) \mapsto (g_0, \dots, g_M)$ is, up to an invertible change of information coordinates, the length-$(M+1)$ single-parity-check code. The two representations above are disjoint recovery sets for each information coordinate. This suggests replacing the one-dimensional telescoping code by a constant-rate code with many geometric recovery sets.

The full construction keeps two features of this example. The resource functions may be arbitrary; the proof uses only their incidence with the recovery sets. Moreover, recovery sets need be disjoint only for requests served in the same group. We next replace the chain by a geometry with many recovery sets, then allow bounded reuse across groups.

4 The local-recovery gadget

Fix a suffix z. We encode the 2^p values $f(a, z)$, as a varies, into one evaluation-codeword. If instead we fix a code coordinate y and vary z, that coordinate becomes a shorter function $G_y(z)$. Local recovery moves across a codeword; recursion evaluates these coordinate functions down the suffix direction. Proposition 4.2 records exactly what the rest of the proof needs from this construction.

Fix an integer $\ell \geq 2$ and a field $\mathbb{F}_q$ of characteristic two, where $q = 2^b$. Set

$$s_0 = \left\lfloor \frac{q-1}{\ell} \right\rfloor$$

and fix a polynomial-basis representation of $\mathbb{F}_q$. If α_i denotes the field element whose canonical b-bit representation is the integer i, choose the efficiently indexed set

$$A = \{\alpha_0, \ldots, \alpha_{s_0-1}\}.$$

Let $\mathcal{V}$ be the space of polynomials

$$P(X_1, \ldots, X_\ell) \in \mathbb{F}_q[X_1, \ldots, X_\ell]$$

having degree less than s_0 in every variable. Repeated univariate interpolation shows that evaluation on A^ℓ uniquely determines P, and

$$\deg P \leq \ell(s_0 - 1) \leq q - 2.$$

The evaluation map

$$\mathrm{Enc} : \mathbb{F}_q^{A^\ell} \longrightarrow \mathbb{F}_q^{\mathbb{F}_q^\ell}, \qquad \big(P(u)\big)_{u \in A^\ell} \longmapsto \big(P(y)\big)_{y \in \mathbb{F}_q^\ell},$$

is a systematic linear code of length

$$N = q^\ell$$

and dimension

$$K = s_0^\ell = \Theta_\ell(q^\ell).$$

4.1 Packing Boolean restrictions

Split an n-bit input as

$$x = (a, z), \qquad |a| = p, \qquad |z| = d = n - p.$$

For each fixed suffix z, pack the 2^p bits

$$\big(f(a, z)\big)_{a \in \{0,1\}^p}$$

into K field symbols, using the fixed $\mathbb{F}_2$-basis of $\mathbb{F}_q$ and padding unused bit positions with zeros. More explicitly, interpret a prefix a as an integer $I(a) \in [2^p]_0$, and set

$$s(a) = \left\lfloor \frac{I(a)}{b} \right\rfloor, \qquad j(a) = I(a) \bmod b.$$

Write $s(a)$ in base s_0 using exactly ℓ digits $i_1(a), \dots, i_\ell(a) \in [s_0]_0$, and define

$$u(a) = (\alpha_{i_1(a)}, \dots, \alpha_{i_\ell(a)}) \in A^\ell.$$

Thus bit $j(a)$ of the information symbol at $u(a)$ is $f(a, z)$. Fixed-divisor division, remainder, and base conversion have Boolean circuits of size $\mathrm{poly}_\ell(p, \log q)$, so computing $(u(a), j(a))$ for all requests costs $\widetilde{O}_\ell(t)$ gates.

Choose q, among powers of two, minimally so that

$$Kb \geq 2^p.$$

For all sufficiently large p (depending on ℓ), the corresponding code dimensions at field sizes q and $q/2$ are both $\Theta_\ell(q^\ell)$. Minimality then gives

$$q^\ell b = \Theta_\ell(2^p). \tag{1}$$

Indeed, the lower bound follows from $Kb \leq q^\ell b$. For the upper bound, the preceding candidate $q/2$ has bit width $b-1$, fails the packing inequality, and has dimension $\Omega_\ell(q^\ell)$. Hence $q^\ell(b-1) = O_\ell(2^p)$; since $b \leq 2(b-1)$ for $b \geq 2$, the upper bound in (1) follows. Equivalently,

$$\ell \log_2 q + \log_2 \log_2 q = p + O_\ell(1), \qquad \log_2 q = \frac{p}{\ell} + O_\ell(\log p). \tag{2}$$

Let P_z be the polynomial whose information symbols are the packed bits. For each code coordinate $y \in \mathbb{F}_q^\ell$, define the field-valued resource

$$G_y(z) = P_z(y).$$

Each of the b coordinate bits of G_y is a Boolean function on d variables, determined by f, y, and the chosen field basis. For $r \in [b]_0$, write

$$H_{y,r}(z) = \big(G_y(z)\big)_r.$$

It is useful to picture the values $P_z(y)$ as a rectangular array. A row, with z fixed and y varying, is a codeword that contains all requested prefix values. A column, with y fixed and z varying, is the resource function G_y on d variables. The line decoder reads selected entries from one row, while the recursive circuit evaluates the required columns.

4.2 Affine-line recovery

Lemma 4.1 (Line identity)**.** *Let $P : \mathbb{F}_q^\ell \to \mathbb{F}_q$ be represented by a polynomial of total degree at most $q-2$. For every $u \in \mathbb{F}_q^\ell$ and every nonzero direction $v \in \mathbb{F}_q^\ell$,*

$$P(u) = -\sum_{\lambda \in \mathbb{F}_q^*} P(u + \lambda v).$$

In characteristic two, the minus sign may be omitted.

Proof. The restriction $Q(\lambda) = P(u + \lambda v)$ is a univariate polynomial of degree at most $q-2$. The sum of the constant monomial over $\mathbb{F}_q$ is $q \cdot 1 = 0$ in characteristic two. For $1 \leq j \leq q-2$, the sum of λ^j over $\mathbb{F}_q^*$ vanishes because j is not divisible by $q-1$, and the zero term contributes nothing. Applying these identities monomial by monomial gives $\sum_{\lambda \in \mathbb{F}_q} Q(\lambda) = 0$. Separating the term $\lambda = 0$ proves the claim. □

Thus every affine line through an information point u supplies a recovery set consisting of its $q-1$ non-target points. We identify two nonzero direction vectors when one is a nonzero scalar multiple of the other and write $[v]$ for the resulting projective direction. The number of such directions through a point is

$$D_q = \frac{q^\ell - 1}{q-1} = 1 + q + \cdots + q^{\ell-1}, \qquad q^{\ell-1} \le D_q < 2q^{\ell-1}. \tag{3}$$

For fixed ℓ and $p \to \infty$, Equations (2) and (3) give the parameter dictionary

$$q^\ell = 2^{p+o(p)}, \qquad q = 2^{p/\ell+o(p)}, \qquad D_q = q^{\ell-1+o(1)} = 2^{((\ell-1)/\ell)p+o(p)}. \tag{4}$$

Two distinct lines through the same target intersect only at that target, which is absent from both recovery sets. Repeated requests for one information symbol therefore cause no additional difficulty.

The facts needed later can now be packaged without reference to the internal polynomial representation.

Proposition 4.2 (Local-recovery gadget)**.** *Fix $\ell \ge 2$ and a split $n = p + d$, with p sufficiently large depending on ℓ, and choose $q = 2^b$ by the packing rule above. For every $f : \{0,1\}^n \to \{0,1\}$ there are $q^\ell b$ Boolean resource functions $H_{y,r} : \{0,1\}^d \to \{0,1\}$ with the following properties.*

(i) *The resource count satisfies $q^\ell b = \Theta_\ell(2^p)$.*

(ii) *A prefix a determines an information point $u(a) \in A^\ell$ and a bit position $j(a) \in [b]_0$ by a circuit of size $\mathrm{poly}_\ell(p, \log q)$.*

(iii) *For every projective direction $[v]$, the set*

$$R(u(a), [v]) = \{u(a) + \lambda v : \lambda \in \mathbb{F}_q^*\}$$

has $q-1$ coordinates and recovers the requested bit by

$$f(a, z) = \bigoplus_{y \in R(u(a),[v])} H_{y,j(a)}(z).$$

There are $D_q = (q^\ell - 1)/(q-1)$ such recovery sets.

(iv) *If $U \subseteq \mathbb{F}_q^\ell$ is already occupied, at most $|U \setminus \{u(a)\}|$ directions have $R(u(a), [v]) \cap U \neq \varnothing$.*

Proof. The packing construction gives (i) and (ii). The line identity, followed by selection of coordinate bit $j(a)$ in the fixed $\mathbb{F}_2$ basis, gives (iii). For (iv), every $y \in U \setminus \{u(a)\}$ lies on exactly one projective line through $u(a)$, while the omitted target $u(a)$ blocks none. □

Thus one desired bit has become $q-1$ requests to shorter resource functions, but those coordinates may be chosen from any of D_q recovery sets. The next section turns the blocking guarantee in (iv) into a deterministic circuit and then routes only the coordinates actually selected.

5 Deterministic scheduling and routing

5.1 Disjoint scheduling inside one group

The geometric count says that a suitable line exists, but a circuit must find one from the target points. We do this greedily. Previously occupied points rule out directions; projective ranking and sorting let an ordinary Boolean circuit choose the first direction that remains.

Consider a multiset of g requested information points $u_1, \dots, u_g \in A^\ell$. Process the requests in order. Suppose disjoint recovery sets have been selected for requests $1, \dots, j-1$, and let U be their union. Then

$$|U| = (j-1)(q-1) < gq.$$

For the target u_j, every point $y \in U \setminus \{u_j\}$ forbids at most one projective direction, namely the direction of $y - u_j$. The point u_j itself forbids none because it is omitted from the new recovery set.

Lemma 5.1 (Projective indexing)**.** *Projective directions in $\mathbb{F}_q^\ell$ have rank and unrank circuits of size* $\mathrm{poly}_\ell(b)$*, with ranks in $[D_q]_0$.*

Implementation idea. The indexing normalizes the first nonzero coordinate to one and orders the normalized vectors by the position of that coordinate and then by their remaining base-q digits. The explicit circuits are given in Section A.1.

Lemma 5.2 (Greedy line scheduler)**.** *If*

$$gq < D_q, \tag{5}$$

then every multiset of g target points admits pairwise disjoint affine-line recovery sets. Given the targets, the recovery sets can be selected by a Boolean circuit of size

$$\widetilde{O}_\ell(g^2 q).$$

Proof idea. At stage j, the blocking property in Proposition 4.2 leaves a valid direction because fewer than D_q directions are forbidden. The circuit ranks the forbidden directions, sorts those ranks, and selects the least missing one; stage j costs $\widetilde{O}_\ell(jq)$. The treatment of duplicate targets, sentinel ranks, and the least-missing-rank circuit is given in Section A.2. Summing over the stages gives the stated cost.

5.2 Request groups

Partition the t requests into C fixed groups, each of size at most

$$g = \left\lceil \frac{t}{C} \right\rceil.$$

Apply Lemma 5.2 independently inside each group. A code coordinate is used at most once per group and therefore at most C times overall. The inequalities $\lceil t/C \rceil \le t/C + 1$ and $2t \le t^2/C + C$, the latter being the arithmetic–geometric mean inequality, show that the total scheduling cost is

$$C \cdot \widetilde{O}_\ell(g^2 q) = \widetilde{O}_\ell\left(\frac{t^2 q}{C} + Cq\right), \tag{6}$$

The construction is valid whenever (5) holds for $g = \lceil t/C \rceil$, that is, whenever

$$\left\lceil \frac{t}{C} \right\rceil q < D_q. \tag{7}$$

The parameter C exposes the central tradeoff. More groups mean fewer requests per group, making disjoint scheduling easier and cheaper. But they also permit each resource coordinate to receive more suffixes, making the recursive C-copy problem harder. The recursion works by finding an exponential choice of C that satisfies both demands. In the proof below, one encoded block supplies the room inside a group and the remaining blocks supply the capacity across groups.

5.3 Scatter and gather

Every non-target point of a selected line produces an incidence record containing a request identifier, a group, a line position, the point's code coordinate y, and the suffix to be evaluated. There are exactly $t(q-1) < tq$ such records. Inside one group, each coordinate appears in at most one record.

The routing consists of four fixed sorting-and-local-update passes. Here "key" means (group, y); any unlisted tie is broken by the record's original position.

Pass	Records	Sort order	Local result
Scatter match	incidence, slot	$(\text{key}, \text{type})$; incidence before slot	a matched slot takes its predecessor's suffix; otherwise it takes a dummy
Scatter order	incidence, filled slot	$(\text{type}, \text{key})$; slots first	slots are in canonical (group, y) order
Gather match	value, saved incidence	$(\text{key}, \text{type})$; value before incidence	a matched incidence takes its predecessor's value
Gather order	value, updated incidence	$(\text{type}, \text{request}, \text{line position})$; incidences first	decoder inputs are in canonical request order

Scatter records carry a d-bit suffix. After resource evaluation, value records carry the b-bit field value $G_y(z)$. The gather passes are payload-agnostic for any payload whose width is polynomial in the displayed parameters.

Lemma 5.3 (Record routing)**.** *The suffixes can be scattered into slots indexed by* $(\text{group}, y) \in [C]_0 \times \mathbb{F}_q^\ell$*, and the resulting resource values can be gathered back to their request records, using circuits of total size*

$$\widetilde{O}_\ell(tq + Cq^\ell).$$

Implementation idea. The four passes in the table use a constant number of sorting networks on $O(tq + Cq^\ell)$ records of polynomial width. The exact fixed-wire treatment of unmatched records and array boundaries is given in Section A.3.

The lemma is the reason no tq^ℓ crossbar appears. Only the actual incidences and one dummy record per resource slot are routed.

6 The composition bound

Bounded-demand invariant. Fix f and the local-recovery gadget above. Each $H_{y,r}$ is one fixed Boolean function on d variables. Inside each request group, disjoint recovery ensures that coordinate y occurs at most once; hence the pair (y, r) receives at most one suffix from each of the C groups. Place those suffixes in the canonical group slots and pad unused slots with a fixed dummy suffix. One circuit for $H_{y,r}^{\times C}$, of size at most $L_C(d)$, serves exactly this bounded demand. There are $q^\ell b$ such pairs, and no sharing between different pairs is assumed.

Proposition 6.1 (Composition bound)**.** *Fix $\ell \geq 2$. Let $n = p + d$ with p sufficiently large depending on ℓ, choose $q = 2^b$ minimally so that $s_0^\ell b \geq 2^p$, and suppose Equation (7) holds. Then every $f : \{0,1\}^n \to \{0,1\}$ satisfies*

$$\mathrm{C}(f^{\times t}) \leq \underbrace{q^\ell b\, L_C(d)}_{\text{recursive resource evaluation}} + \widetilde{O}_\ell\left(\underbrace{\frac{t^2 q}{C}}_{\text{line scheduling}} + \underbrace{tq}_{\text{incidences and decoding}} + \underbrace{Cq^\ell}_{\text{resource slots}} \right). \quad (8)$$

The first term is recursive: there are $q^\ell b$ Boolean resource functions, each evaluated on at most C suffixes of length d. The other three terms account for scheduling, the selected line incidences, and the padded resource slots. Increasing C makes the groups smaller, but gives every resource more suffixes to process. The induction chooses C so that the recursive term stays at scale $2^n/n$ and all three bookkeeping terms are exponentially smaller.

Proof. Map every requested prefix a to its information point $u(a)$ and bit position $j(a)$ using the canonical packing circuit. In each request group, use Lemma 5.2 to select and enumerate a recovery line. Together these steps cost $\widetilde{O}_\ell(t) + \widetilde{O}_\ell(t^2 q/C + Cq)$ gates. Each non-target point y on a selected line requests G_y on the suffix of the corresponding output instance.

For each pair (y, r), use the $H_{y,r}^{\times C}$ circuit from the bounded-demand invariant on the canonical group slots. The definition of $L_C(d)$ applies separately to each induced function $H_{y,r}$, so these circuits cost $q^\ell b L_C(d)$ in total. The complete scatter, gather, and fixed-wire compaction pipeline costs $\widetilde{O}_\ell(tq + Cq^\ell)$ by Lemma 5.3. Field addition is coordinatewise XOR in the fixed $\mathbb{F}_2$ basis, so XORing the $q - 1$ returned b-bit values of a request and selecting bit $j(a)$ costs $O(qb)$ per request and $O(tqb)$ in total. Since each selected line passes through its target $u(a)$ and $\deg P_z \leq q - 2$, Lemma 4.1 shows that this XOR equals $P_z(u(a))$ for the request's suffix z, and bit $j(a)$ of that symbol is $f(a, z)$ by the packing of Section 4.1. The circuit therefore computes $f^{\times t}$.

The polynomial P_z and the functions G_y specify the nonuniform resource truth tables; no run-time encoder is being assumed. Since $q \geq 2$, all smaller terms in this gate ledger are absorbed by the overhead in (8), proving the bound. □

7 The equal-block induction

We now apply the composition bound recursively. At level k, split the input into $k+1$ approximately equal blocks, encode one block, and invoke the preceding level on the remaining k.

For an integer $k \geq 1$, let P_k denote the assertion that, for every fixed $0 \leq \gamma < k/(k+1)$, there is a constant $A_{k,\gamma}$ such that, for all sufficiently large n, every $f : \{0,1\}^n \to \{0,1\}$, and every integer $1 \leq t \leq 2^{\gamma n}$,

$$\mathrm{C}(f^{\times t}) \leq A_{k,\gamma} \frac{2^n}{n}.$$

Every comparison below has a fixed positive linear margin in its base-two logarithm. We use without further comment that such a margin absorbs the $o(n)$ terms, every fixed polynomial factor hidden by $\widetilde{O}$, and the $O(1)$ changes caused by floors and ceilings.

7.1 Two equal blocks: the base case

Lemma 7.1 (Two-block base)**.** P_1 *holds.*

Here $C = 1$, so no mass-production hypothesis is used inside a resource. The two equal blocks make the resource term proportional to $2^{n/2}L(n/2)$; the one-half threshold comes from the quadratic scheduler term $t^2 q$.

Proof. Fix $0 \leq \gamma < 1/2$, let $\eta = 1/2 - \gamma > 0$, and suppose $t \leq 2^{\gamma n}$. Put

$$m = \left\lfloor \frac{n}{2} \right\rfloor, \qquad p = m, \qquad d = n - m, \qquad C = 1.$$

Thus $n = 2m + O(1)$ and all recovery sets will be globally disjoint. In block units, the request and code parameters are

$$\log_2 t \leq m - \eta n + O(1), \qquad \log_2 q = \frac{m}{\ell} + o(n), \qquad \log_2 D_q = \left(1 - \frac{1}{\ell}\right) m + o(n).$$

Choose a sufficiently large fixed ℓ so that

$$\frac{1}{\ell} < \eta. \tag{9}$$

Since $2m/\ell \leq n/\ell < \eta n$, this gives

$$m - \eta n + \frac{m}{\ell} < m - \frac{m}{\ell}.$$

The strict linear margin now implies $tq < D_q$ for all sufficiently large n.

The resource term in Equation (8) is, by Shannon–Lupanov synthesis,

$$q^\ell b L(d) = O_\ell\left(2^p \frac{2^d}{d}\right) = O_\gamma(2^n/n).$$

The scheduler exponent is

$$\log_2(t^2 q) \leq 2m - 2\eta n + \frac{m}{\ell} + o(n) < n,$$

where the strict inequality follows from (9). Likewise, $\log_2(tq) \leq m - \eta n + m/\ell + o(n) < n$ and $\log_2 q^\ell = m + o(n) < n$. After the suppressed polynomial factors are restored, every overhead term is $2^{(1-\Omega_\gamma(1))n}$ and is negligible compared with $2^n/n$. The same strict margins absorb all rounding. □

7.2 Adding one equal block

Lemma 7.2 (Equal-block step)**.** *For every integer $k \geq 2$, P_{k-1} implies P_k.*

Proof. Fix $0 \leq \gamma < k/(k+1)$. If $\gamma < (k-1)/k$, the assertion is already P_{k-1}. We may therefore assume

$$\frac{k-1}{k} \leq \gamma < \frac{k}{k+1}.$$

Let

$$\eta = \frac{k}{k+1} - \gamma > 0, \qquad m = \left\lfloor \frac{n}{k+1} \right\rfloor, \qquad p = m, \qquad d = n - m.$$

Write $n = (k+1)m + r$ with $0 \leq r \leq k$, and choose the number of request groups to be

$$C = \left\lceil 2^{(k-1)m - \eta n/2} \right\rceil. \tag{10}$$

This choice splits the available slack in half. Dividing the requests among C groups puts each group $\eta n/2$ below the one-block exponent m; measured on the remaining d bits, the exponent of C is also a fixed amount below the threshold supplied by P_{k-1}. The assumption $\gamma \geq (k-1)/k$ ensures that the exponent in (10) is positive for all sufficiently large n.

Let $\beta = (k-1)/k$. Since $d = km + r$,

$$\beta d - \log_2 C = \frac{\eta n}{2} + \frac{k-1}{k} r + O(1).$$

In particular, $C \leq 2^{(\beta - \eta/4)d}$ for all sufficiently large n. The exponent $\beta - \eta/4$ is fixed and strictly below the threshold for P_{k-1}, so the inductive hypothesis gives

$$L_C(d) = O_{k,\gamma}(2^d/d).$$

By Equation (1), the resource term is therefore

$$q^\ell b L_C(d) = O_{k,\gamma,\ell}\left(2^m \frac{2^d}{d}\right) = O_{k,\gamma,\ell}(2^n/n).$$

The remaining estimates are easiest to read in block units. Since $\log_2 t \leq \gamma n = km - \eta n + O_k(1)$, the size $g = \lceil t/C \rceil$ of a request group satisfies

$$\log_2 g \leq m - \frac{\eta n}{2} + O_k(1).$$

The code parameters satisfy

$$\log_2 q = \frac{m}{\ell} + o(n), \qquad \log_2 D_q = \left(1 - \frac{1}{\ell}\right) m + o(n).$$

Choose a sufficiently large fixed ℓ so that

$$\frac{2}{(k+1)\ell} < \frac{\eta}{2}. \tag{11}$$

Then $2m/\ell < \eta n/2$, and hence

$$m - \frac{\eta n}{2} + \frac{m}{\ell} < m - \frac{m}{\ell}.$$

Together with (3), this is exactly the strict exponent margin required for Equation (7).

Finally, the complete overhead ledger in Equation (8) is

$$\log_2\left(\frac{t^2q}{C}\right) \le (k+1)m - \frac{3\eta n}{2} + \frac{m}{\ell} + o(n) < n,$$
$$\log_2(tq) \le n - m - \eta n + \frac{m}{\ell} + o(n) < n,$$
$$\log_2(Cq^\ell) \le n - m - \frac{\eta n}{2} + o(n) < n.$$

The first strict inequality uses $(k+1)m \le n$ and (11); the other two use $m = \Theta_k(n)$ and $\ell > 1$. Thus every overhead term is exponentially smaller than $2^n/n$. This proves P_k. □

Proof of Theorem 1.1. By Lemmas 7.1 and 7.2, P_k holds for every $k \ge 1$. Fix $0 \le \gamma < 1$ and choose k so that $\gamma < k/(k+1)$. Then P_k gives the asserted bound beyond a threshold N_γ. For the finitely many integers $1 \le n < N_\gamma$, use $\mathrm{C}(f^{\times t}) \le t\mathrm{C}(f)$ and enlarge A_γ to dominate $nt\mathrm{C}(f)/2^n$ over all relevant n, f, and t. This proves the statement for every $n \ge 1$. □

8 Implications and open problems

The theorem settles the fixed-rate question: every fixed $\gamma < 1$ is achievable at the optimal worst-case order. The two immediate quantitative questions are whether recursion is necessary and where the plateau ends as the copy exponent approaches one.

Can recursion be removed? The disjoint base case isolates the reason for the recursion. With $C = 1$ and $d = \delta n$, the resource term remains $O(2^n/(\delta n))$, and for large fixed ℓ there are enough directions whenever $\gamma < 1 - \delta$. It is the quadratic scheduling term t^2q, not line availability, that restricts this argument to $\gamma < 1/2$. Grouping divides that term by C, while recursion pays for the resulting C simultaneous uses of each resource. At the exponent level, this is the update $\beta \mapsto 1/(2-\beta)$ underlying the equal-block sequence. A disjoint-line scheduler of size $\widetilde{O}(tq)$ would remove the need for recursion: for any fixed $\gamma < 1$, one could choose $0 < \delta < 1 - \gamma$ and take $C = 1$. Finding such a scheduler, or ruling one out, is therefore a natural next problem.

Where does the plateau end? The theorem reaches $2^{(1-\varepsilon)n}$ copies for every fixed $\varepsilon > 0$, but does not treat a rate that approaches one with n. Some degradation in that regime is forced. If $n \ge 2$ and f depends on all n of its variables, every output of a circuit for $f^{\times t}$ is a gate, and each of the tn inputs must feed at least one gate, since otherwise the corresponding output would not depend on it. Since gates have fan-in at most two,

$$\mathrm{C}(f^{\times t}) \ge \frac{tn}{2}.$$

A bound of the form $A\,2^n/n$ therefore cannot hold once $t > 2A \cdot 2^n/n^2$; in particular, a uniform $O(2^n/n)$ plateau must fail for $t = \omega(2^n/n^2)$. The theorem still leaves the detailed mass-production curve throughout the near-endpoint regime $t = 2^{n-o(n)}$ unresolved. Determining how far the plateau extends, and then characterizing the growth beyond it, would require a quantitatively sharper schedule or a different recursive decomposition.

A Circuit implementation details

This appendix records the gate-level details behind the scheduler and routing lemmas. The main proof uses only their statements and costs.

A.1 Projective-direction indexing

Proof of Lemma 5.1. Normalize a nonzero vector so that its first nonzero coordinate is one. The normalized directions whose first nonzero coordinate is in position $h \in \{1, \dots, \ell\}$ have the form

$$(0, \dots, 0, 1, w_{h+1}, \dots, w_\ell)$$

and form a block of size $q^{\ell-h}$. Let

$$B_h = \sum_{r=1}^{h-1} q^{\ell-r}, \qquad \mathrm{val}_q(w_{h+1}, \dots, w_\ell) = \sum_{r=h+1}^{\ell} \mathrm{int}(w_r) q^{\ell-r},$$

where $\mathrm{int}(w_r) \in [q]_0$ is the canonical binary representation. The rank is

$$B_h + \mathrm{val}_q(w_{h+1}, \dots, w_\ell).$$

The blocks are consecutive and their total size is D_q.

To rank, find the first nonzero coordinate, invert it, normalize the vector, and concatenate the b-bit tail coordinates. To unrank, compare the rank with the ℓ hardwired block intervals, subtract the appropriate B_h, and split the remainder into base-q digits. Since $q = 2^b$, the last step is bit slicing. The field operations and comparisons from the circuit conventions give size $\mathrm{poly}_\ell(b)$ in both directions. □

A.2 The least-missing-direction circuit

Proof of Lemma 5.2. At step j, fewer than D_q directions are forbidden, so a valid direction exists. This proves the combinatorial statement.

For the circuit bound, represent a projective direction by normalizing its first nonzero coordinate to one and use the ranking from Lemma 5.1. At stage j, for each $y \in U$, output the sentinel rank D_q when $y = u_j$ and otherwise output the projective rank of $y - u_j$. At the gate level, first multiplex a fixed nonzero vector in place of $y - u_j$ when $y = u_j$, rank that always-nonzero vector, and then overwrite the result by D_q on the equality branch. Use $\lceil \log_2(D_q + 1) \rceil$ bits per rank. There are $(j-1)(q-1) = O(jq)$ such entries.

Sort these ranks as $r_1 \le \cdots \le r_s$. The least missing rank can be found without a dense D_q-bit table. Rank zero is a candidate if $s = 0$ or $r_1 > 0$; after position i, rank $r_i + 1$ is a candidate if

$$r_i < D_q - 1 \quad \text{and} \quad (i = s \text{ or } r_{i+1} > r_i + 1).$$

These tests are applied to the full sorted list, including duplicates and sentinels. Only the last occurrence of a non-sentinel rank can generate its successor, and a sentinel cannot generate a candidate.

A left-to-right prefix-OR circuit identifies the first candidate in this order. Multiplexers select its rank using $O(s \log D_q)$ gates.

Since $|U \setminus \{u_j\}| < D_q$, fewer than D_q distinct non-sentinel ranks are forbidden, so a candidate exists. Sorting, comparisons, and priority selection therefore cost $\widetilde{O}_\ell(jq)$ gates. Unrank the selected direction using Lemma 5.1 and enumerate its $q-1$ non-target line points, at cost $q\,\mathrm{poly}_\ell(b)$. Unrolling the stages and summing over $j \le g$ gives $\widetilde{O}_\ell(g^2 q)$. □

A.3 Four-pass fixed-wire routing

Proof of Lemma 5.3. For scatter, create one slot record for each of the Cq^ℓ keys and combine them with the fewer than tq incidence records. Preserve a copy of every incidence record by free fan-out. Uniqueness inside a group puts at most one incidence at each key. After the first pass in the table, a slot therefore copies the preceding suffix exactly when a type-and-key equality test succeeds; otherwise it receives a fixed dummy suffix. The predecessor of the first position is defined to be a dummy record, so the boundary case uses the same guard. The second pass places the filled slots in canonical (group, y) order. Fixed wiring transposes this order to (y, group), the input order used by the resource circuits.

For gather, combine the preserved incidences with one value record per key. The third pass places that value immediately before every matching incidence; a guarded local multiplexer copies it into the incidence. The fourth pass puts the first $t(q-1)$ positions in the fixed request-and-line order required by the decoders. Thus absent incidences, the first array position, and all type boundaries are handled by fixed dummy data and the same guarded update, not by data-dependent wiring.

This is a constant number of sorting networks on $O(tq + Cq^\ell)$ records. Each record has width $\mathrm{poly}_\ell(n, \log q, \log t, \log C)$, so the standard primitives from the circuit conventions give the stated bound. □

References

[1] Andreas Albrecht. On simultaneous realizations of Boolean functions, with applications. In *Parcella '88: Proceedings of the Fourth International Workshop on Parallel Processing by Cellular Automata and Arrays*, LNCS 342, pages 51–56. Springer, 1989. doi:10.1007/3-540-50647-0_102.

[2] Kenneth E. Batcher. Sorting networks and their applications. In *Proceedings of the 1968 Spring Joint Computer Conference*, AFIPS Conference Proceedings 32, pages 307–314, 1968. doi:10.1145/1468075.1468121.

[3] Arman Fazeli, Alexander Vardy, and Eitan Yaakobi. PIR with low storage overhead: Coding instead of replication. arXiv:1505.06241, 2015. https://arxiv.org/abs/1505.06241.

[4] Gudmund Skovbjerg Frandsen and Peter Bro Miltersen. Reviewing bounds on the circuit size of the hardest functions. *Information Processing Letters*, 95(2):354–357, 2005. doi:10.1016/j.ipl.2005.03.009.

[5] Lukas Holzbaur, Rina Polyanskaya, Nikita Polyanskii, and Ilya Vorobyev. Lifted Reed–Solomon codes with application to batch codes. In *2020 IEEE International Symposium on Information Theory*, pages 634–639. IEEE, 2020. doi:10.1109/ISIT44484.2020.9174402.

[6] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Batch codes and their applications. In *Proceedings of the 36th Annual ACM Symposium on Theory of Computing*, pages 262–271. ACM, 2004. doi:10.1145/1007352.1007396.

[7] William Kretschmer. Quantum mass production theorems. In *18th Conference on the Theory of Quantum Computation, Communication and Cryptography*, LIPIcs 266, Article 10, pages 10:1–10:11, 2023. doi:10.4230/LIPIcs.TQC.2023.10.

[8] Oleg B. Lupanov. On a method of circuit synthesis. *Izvestiya VUZ, Radiofizika*, 1(1):120–140, 1958. In Russian. https://radiophysics.unn.ru/issues/1958/1/120.

[9] Wolfgang J. Paul. Realizing Boolean functions on disjoint sets of variables. *Theoretical Computer Science*, 2(3):383–396, 1976. doi:10.1016/0304-3975(76)90089-X.

[10] Claude E. Shannon. The synthesis of two-terminal switching circuits. *Bell System Technical Journal*, 28(1):59–98, 1949. doi:10.1002/j.1538-7305.1949.tb03624.x.

[11] Dietmar Uhlig. On the synthesis of self-correcting schemes from functional elements with a small number of reliable elements. *Mathematical Notes of the Academy of Sciences of the USSR*, 15(6):558–562, 1974. doi:10.1007/BF01152835.

[12] Dietmar Uhlig. Networks computing Boolean functions for multiple input values. In M. S. Paterson, editor, *Boolean Function Complexity*, London Mathematical Society Lecture Note Series 169, pages 165–173. Cambridge University Press, 1992. doi:10.1017/CBO9780511526633.013.

[13] Ingo Wegener. *The Complexity of Boolean Functions*. Wiley–Teubner, 1987. https://eccc.weizmann.ac.il/static/books/The_Complexity_of_Boolean_Functions/.